Leapfrog Assignment 2: A Retrieval Augmented Generation (RAG) Pipeline for LF Job Descriptions

Raag Yatchu Maharjan

Leapfrog Technology Inc., KTM

Abstract

This report documents the design, implementation, and evaluation methodology of a Retrieval-Augmented Generation system built for natural-language job search over a structured CSV dataset of 1,000 job listings. The system takes a user query, retrieves semantically relevant job description chunks from a locally persisted ChromaDB vector store, and passes those chunks as grounded context to a local language model that produces a cited, concise answer. The ingestion pipeline cleans HTML job descriptions, splits them into overlapping chunks, and embeds them using the BAAI/bge-small-en-v1.5 sentence transformer. At query time, a hybrid retriever combines dense vector search with keyword scoring, fuses both result sets through Reciprocal Rank Fusion, and reranks candidates with a cross-encoder before prompt construction. The language model backend is Hugging Face Transformers running Qwen2.5-3B-Instruct locally. The system is served through a FastAPI application with Pydantic request validation and structured JSON logging. This report aims to cover the full system architecture, each component in the data flow, prompt design decisions, engineering considerations, evaluation metrics, known limitations, and recommended future improvements.

***K*eywords:** *Retrieval-Augmented Generation, Vector Search, ChromaDB, Hugging Face Transformers, FastAPI, Pydantic, Reranker*

1 Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

RAG or Retrieval-Augmented Generation is an architectural paradigm in natural language processing that combines information retrieval with text generation. A standalone language model answers from its learned internal parameters alone, which means that it relies on knowledge encoded dyring the training phase plus the context provided by a user prompt during that time. This can lead to real practical issues, it can hallucinate information, fabricate plausibl souding details when its parametric memeory is insufficient. Standards language models are also incapable of fitting a large dataset into every prompt.

A RAG system addressses these issues by retrieving relevant external context at query time and injecting the context into the model prompt. The model is then instructed to answer only from the retrieved context.

For the context of this project, the external knowledge base is a CSV of job listings provided by Leapfrog Technology Inc. The retrieval layer finds matching job description chunks, the generation layer uses those chunks to produce a job recommendation based on the user query.

1.2 Purpose of the System

The job RAG Pipeline loads jobs from a CSV file, cleans and normalizes the HTML formatted job descriptions, converts them into searchable text chunks, embeds those chunks into vector representations, stores the vectors in a local ChromaDB vector database, and injects the language model with context at query time for a more grounded response.

1.3 Objectives

My objectives of this assignment are to:

1. Design and implement a RAG pipeline that can effectively retrieve relevant job descriptions based on user queries.
2. Evaluate the performance of the retrieval and generation components using appropriate metrics.
3. Document the entire process, including the methodology, results, and insights gained from the project.
4. Optionally try to implement a reranker model to further improve the retrieval performance of the system.
5. Identify limitations of the current system and propose potential improvements for future iterations.

2 Methodology

This section outlines the methodology I followed to design, implement, and evaluate the RAG system for job descriptions. The process can be broadly categorized into several phases, starting with data exploration and ingestion, followed by retrieval, reranking, generation, and finally serving the application (client).

Before diving into the details of each phase, it is important to understand the structure of the dataset and the challenges it presents. The provided dataset LFJobs.csv consists of 1,000 job listings with the following columns:

- **ID:** A unique identifier for each job listing.
- **Job Category:** The category or field of the job.
- **Job Title:** The title of the job position.

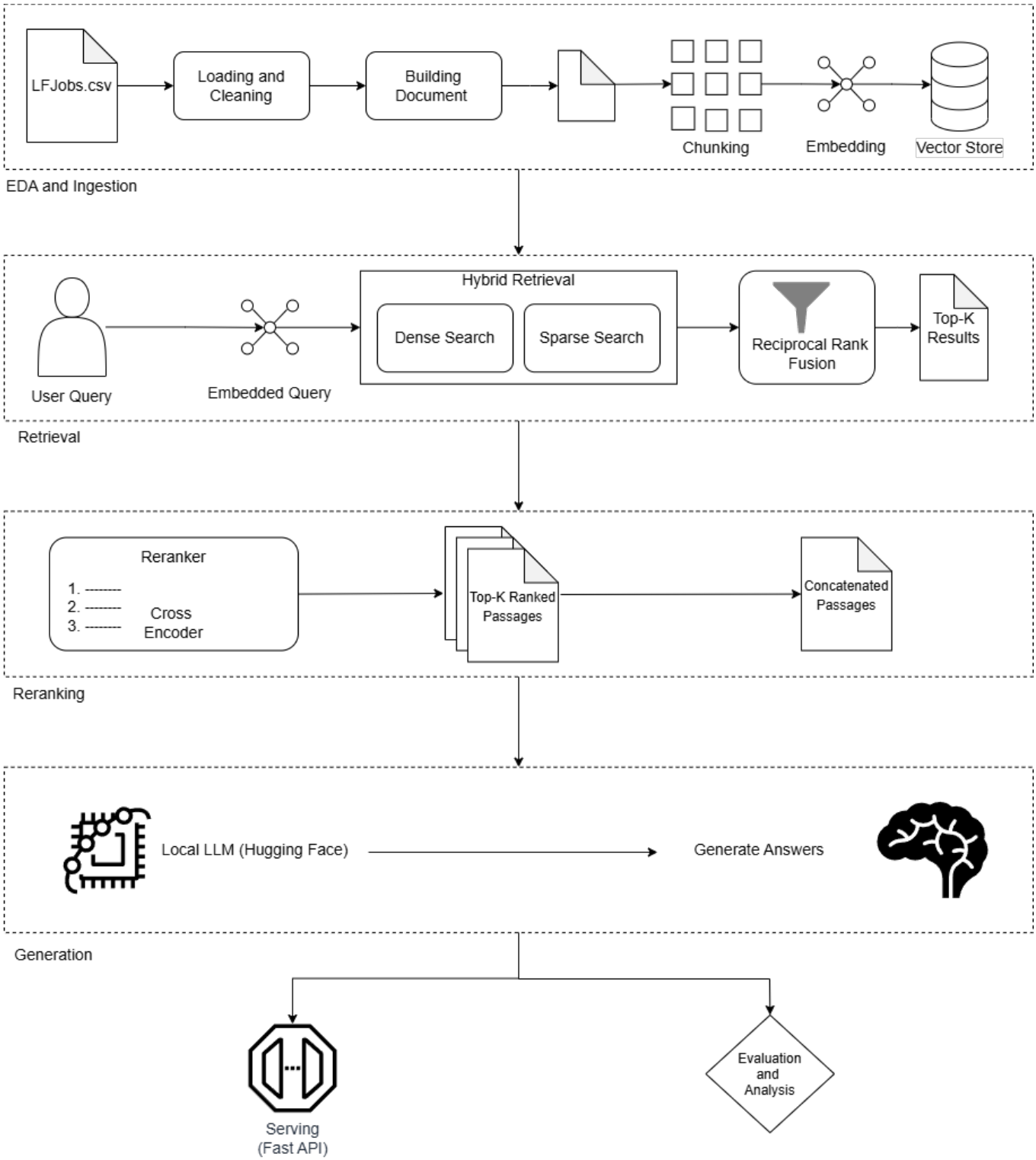

Figure 1: Process Flowchart of the RAG System

- **Company Name:** The name of the company offering the job.
- **Publication Date:** The date when the job listing was published.
- **Job Location:** The location where the job is based.
- **Job Level:** The seniority level of the job (e.g., entry-level, mid-level, senior).
- **Tags:** Keywords or tags associated with the job listing.
- **Job Description:** A detailed description of the job responsibilities, requirements, and other relevant information.

The "Job Description" column contains HTML formatted text, which requires cleaning and normalization before it can be used for retrieval and generation. Normalization involves removing HTML tags, special characters, and extra whitespace, as well as converting the text to lowercase for consistency.

The entire methodology of the project can be broadly categorized into 5 main phases:

1. **EDA and Data Ingestion:** Cleaning data, building documents, chunking, embedding, and storing to ChromaDB.
2. **Retrieval:** Using a hybrid retriever to find relevant job description chunks based on user queries, combine the results using reciprocal rank fusion.
3. **Reranking:** Reranking the retrieved chunks using a cross-encoder to improve the relevance of the context provided to the language model.
4. **Generation:** Constructing prompts for the LLM as a wrapper around the retrieved context and generating responses to user queries.
5. **Serving:** Building a FastAPI application to serve the RAG system and handle user queries in real-time.
6. **Output and Evaluation:** Evaluating the performance of the retrieval and generation components using appropriate metrics and documenting the results.

The above methodology is depicted in the process flowchart in Figure 1.

2.1 EDA and Data Ingestion Phase

Data ingestion refers to the entire process of taking raw data from the source (in this case, the LFJobs.csv file), processing it to make it suitable for retrieval and generation, and storing it in a format that can be efficiently accessed by the RAG system. This ingestion pipeline was first trialed and tested through a Jupyter notebook, and then implemented in a Python script for the final system.

Exploratory Data Analysis and Data Preprocessing

Exploratory Data Analysis (EDA)

EDA (Exploratory Data Analysis) refers to the initial phase of data analysis where we explore the dataset to understand its structure, identify patterns, and detect anomalies.

For this assignment, I performed EDA on the LFJobs.csv dataset to gain insights into the data and inform the subsequent steps of the RAG pipeline.

First, I loaded the dataset and examined the distribution of the job categories and job levels. The distributions were as follows:

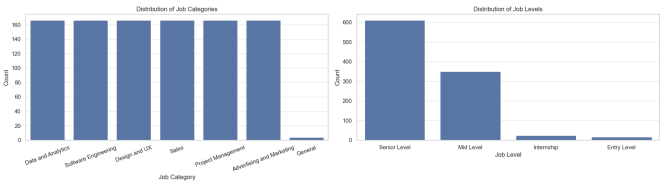

Figure 2: Distribution of Job Categories and Job Levels in the LFJobs Dataset

As seen in this above chart, the distributions of job categories are relatively balanced across the various categories. With similar counts (160 - 170) for all the categories except the "General" category.

The general category has significantly lower count (below 20) which suggests weaker semantic representation of general jobs.

The distribution of job levels is heavily skewed towards the senior level positions, with over 600 listings for it.

Followed by mid-level positions with around 300 listings, and entry-level and internship positions with less than 50 listings. This suggests that this dataset could pose a retrieval bias towards the senior position over the other levels.

I also performed text analysis on the job descriptions to understand the average length of the descriptions and the presence of HTML tags.

As shown in Figure 3, most HTML descriptions range from 3,500–8,000 chars, some exceed 15,000 chars, and the average length is around 6,000 chars. Raw HTML is too noisy and too large for direct embedding, so cleaning and chunking are necessary steps.

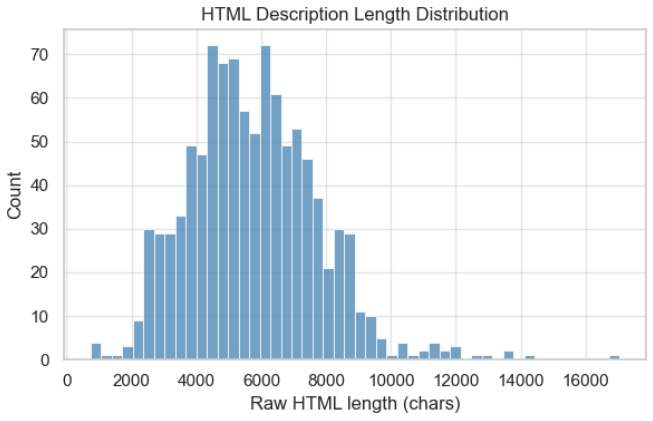

Figure 3: Length of the Raw Job Descriptions in LFJobs Dataset

So, as a data prep step, I cleaned the HTML tags and normalized the text in the job descriptions to make them more suitable

for embedding and retrieval. After cleaning, the average length of the job descriptions reduced to around 2,500 to 8,000 chars, with most descriptions falling between 2,000–6,000 chars as shown in Figure 4.

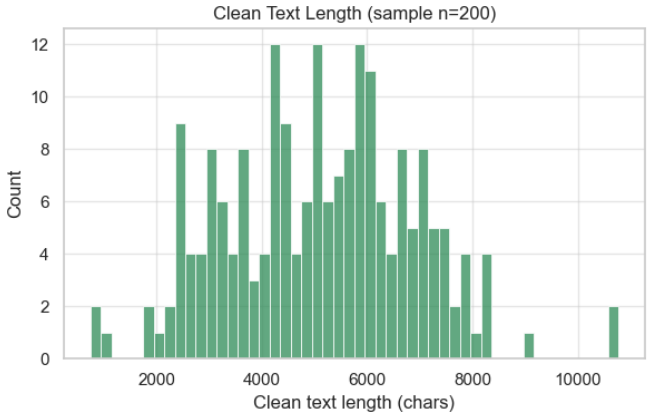

Figure 4: Length of the Cleaned Job Descriptions in LFJobs Dataset

The above figures shows a clean text length sample (200) distribution. We can see that the cleaned descriptions are still quite long, which reinforces the need for chunking to fit within the context window of the language model.

After cleaning and normalizing the job descriptions, I proceeded to build documents for each job listing, which consist of the cleaned description along with relevant metadata such as job title, company name, and job category. This metadata can be useful for retrieval and generation, as it provides additional context about the job listings and can help the retriever find more relevant chunks based on user queries that may reference specific job titles

Data Chunking and Embedding

Given the length of the cleaned job descriptions, I implemented a chunking strategy to split each description into smaller, overlapping chunks. Entire document chunking would dilute the semantic relevance of the retrieved context, while chunks too small would not solve the issue of large descriptions.

Hence, a recursive chunking strategy would be ideal as it would create chunks of varying sizes, ensuring each description is represented by multiple chunks that capture different levels of granularity. Figure 5 illustrates the state of the chunks after the chunking process.

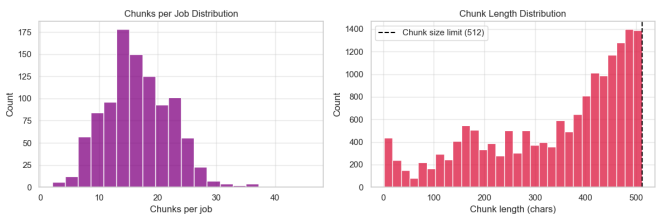

Figure 5: Statistics of the Job Description Chunks after Chunking Process

Since, information spans across chunk boundaries, an overlap of 64 characters was implemented to preserve semantic continuity between adjacent chunks. This allows the retriever to capture relevant information that may be split across chunks.

As seen in the Figure 5, most jobs generated about 15-25 chunks. Some even exceed 40, which indicates that large jobs may dominate the retrieval results if not properly managed.

The chunk length distribution shows that most chunks are close to the 512 characters limit, which indicates that the chunking strategy is effective and efficient. This limit was obtained after much hit and trial. The length needed to be long enough to capture semantic meaning, but short enough to fit within the context window of the language model when combined with other chunks.s

Finally, visualizing and analyzing the number of companies with listings in the dataset. It was observed that that most of the listings where dominated by a few companies, with the top 20 companies accounting for over 60% of the listings as shown in Figure 6. As shown in figure 6, this suggests that the dataset is biased towards these companies.

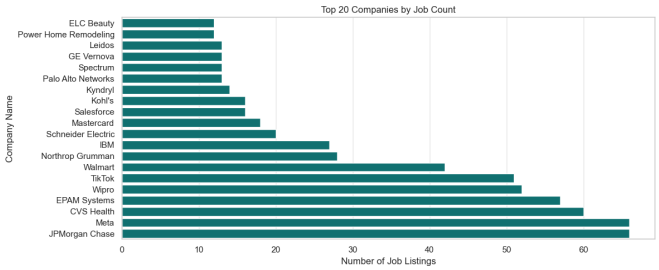

Figure 6: Distribution of Job Listings by Company in LFJobs Dataset

After the EDA and data prep stage, the following insights were concluded:

- The dataset is relatively balanced across job categories but heavily skewed towards senior-level positions
- Job descriptions are long and contain HTML, necessitating cleaning and chunking for effective retrieval
- A recursive chunking strategy with overlap is effective in preserving semantic continuity while fitting within
- The dataset is dominated by a few companies, which may introduce bias in retrieval results

These biases in dataset requires careful consideration as the retrieved results may be skewed towards certain categories, levels, or companies, which could affect the relevance and diversity of the generated responses.

Hence, implementing a hybrid retriever that combines dense and sparse retrieval methods, along with a reranker, can help mitigate these biases and improve the overall performance of the RAG system.

Also metadata filtering and diversification strategies can be implemented in the future to further address these biases.

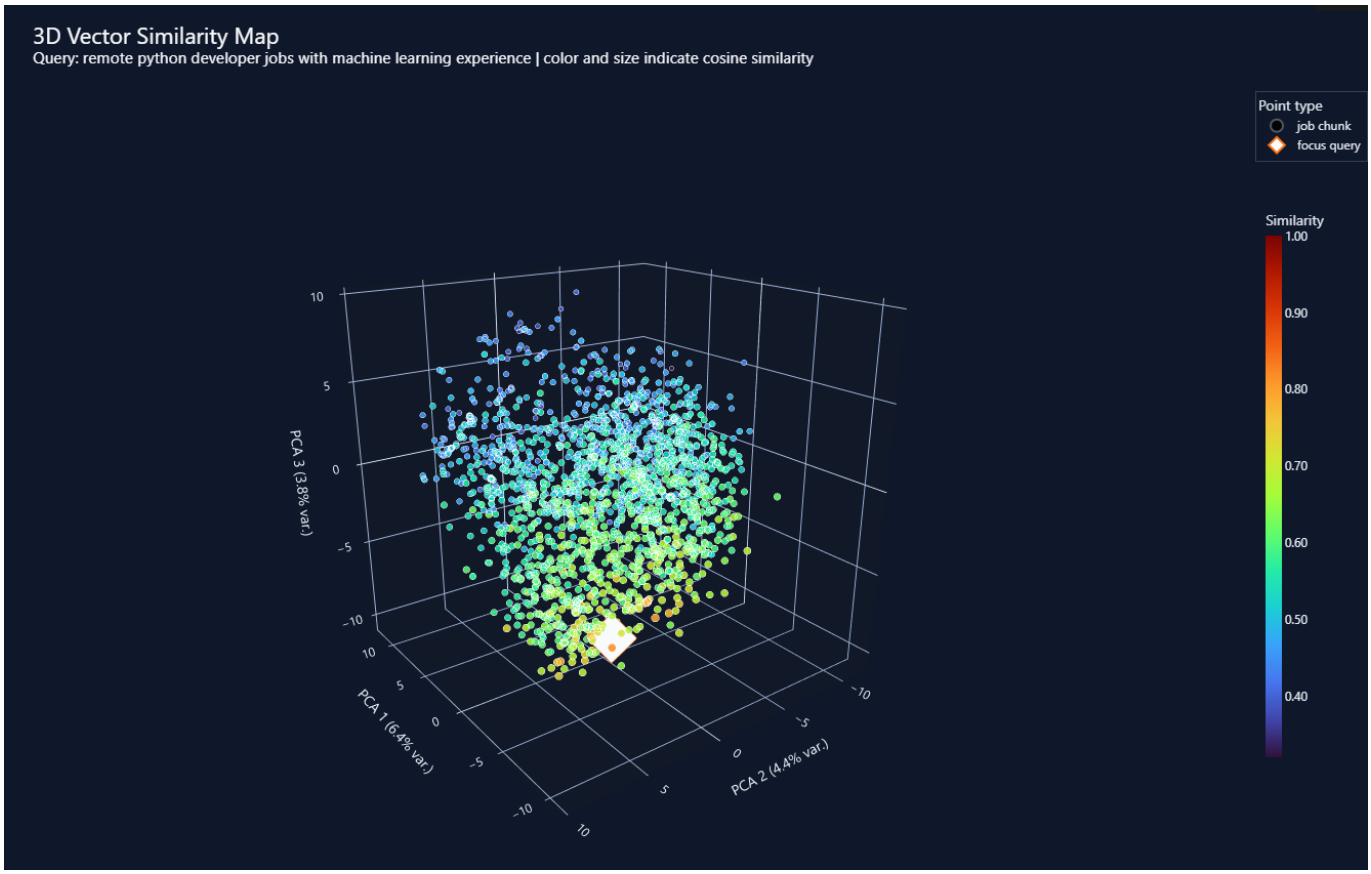

Figure 7: 3D Visualization of the Embedding Space

Embedding the Chunks

Embedding refers the process of converting text into dense vector representations that capture the semantic meaning of the text on a continuous vector space. This multidimensional representation allows for more effective similarity comparisons between text chunks and user queries, which is essential for retrieval in a RAG system.

For this project, I used the BAAI/bge-small-en-v1.5 sentence transformer model to embed the job description chunks. This model is designed for generating high-quality sentence embeddings that capture semantic meaning effectively, making it suitable for our retrieval task.

The BAAI/bge-small-en-v1.5 model is a bidirectional, hugging face transformer model optmized for generting sentence embeddings. It is a small light weight and efficient model with about 33.4 million parameters. This model was chosen for its balance between perfomance and computational efficiency, which is important for this locally served RAG system.

The context window of this model is also algns with our chunk limit of 512 characters, ensuring that each chunk can be fully embedded without truncation. The embedded vectors space can be visualzied with a 3 dimensional PCA plot as shown in Figure 7.

PCA (Principal Component Analysis) is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible. By applying PCA to the embedded vectors, we can visualize the semantic relationships between the job description chunks in a 3D space. Clusters of points in this space may indicate semantically similar chunks, while the distance between points can reflect their semantic dissimilarity.

Vector Store

The generated embeddings were stored in a local ChromaDB vector database, which provides efficient storage and retrieval of high-dimensional vectors.

ChromaDB is an open-source vector database that supports fast similarity search and is optimized for use cases like RAG systems.

It allows for efficient indexing and retrieval of the embedded job description chunks based on their semantic similarity to user queries. The vector store was configured to use cosine similarity as the distance metric for retrieval, which is commonly used for measuring the similarity between dense vector representations in NLP tasks.

The architecture of ChromaDB consists of these main components as shown in Figure 8:

Vector Store: Responsible for storing the embedded vectors and their associated metadata, allowing for efficient retrieval based on similarity.

Core Chroma Engine: Handles the core functionalities of the database, including vector indexing, search algorithms, and data management.

Query Processor: Optimizing and executing search queries against the vector store, utilizing the core engine to retrieve relevant results based on cosine similarity.

API Layer: Provides an interface for interacting with the database, allowing for operations such as inserting new vectors, querying for similar vectors, and managing the stored data.

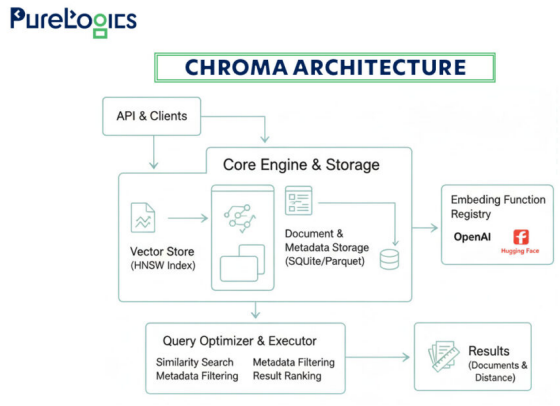

Figure 8: Architecture of ChromaDB Vector Database
Source: https://purelogics.com/what-are-vector-databases/

Cosine Similarity measures the cosine of the angle between two vectors in a multi-dimensional space, providing a value between -1 and 1 that indicates how similar the two vectors are in terms of their direction.

A value of 1 indicates that the vectors are identical in direction, while a value of 0 indicates that they are orthogonal (i.e., have no similarity), and a value of -1 indicates that they are diametrically opposed.

Mathematically, cosine similarity between two vectors A and B can be calculated as follows:

$$CosineSimilarity = \frac{A \cdot B}{||A||||B||} \tag{i}$$

Where:

- $A \cdot B$ is the dot product of vectors A and B
- $||A||$ and $||B||$ are the magnitudes (norms) of vectors A and B, respectively

ChromaDB was picked over other vector databases like FAISS or other vector store libraries due to its ease of use, efficient indexing and retrieval capabilities.

It is simple to setup and run locally, persistant without external dependencies, python native and sutiable for both notebook and script usages.

It also provides a higher level document store abstraction with metadata support, which is useful for our use case.

2.2 Retrieval Phase

Retrieval refers to the process of finding relevant job description chunks from the vector store based on a user query. There are many different retrieval startegies that can be implemented in a RAG system, each with its own advantages and disadvantages.

As we explored and analysed during the phase, implementing a hybrid retriever that combines dense vector search and sparse keyword search to improve the relevance of the retrieved results.

This is becase the dataset was found to be heavily skewed towards certain categories, levels, and companies, which could introduce bias in the retrieval results if only one retrieval method was used. The search results from both methods are then combined using Reciprocal Rank Fusion (RRF) to create a more robust and comprehensive set of candidate chunks for the generation phase.

Before retrieval, the user query is also embedded using the same BAAI/bge-small-en-v1.5 model to ensure that the query and the job description chunks are represented in the same vector space for effective similarity comparison.

Hybrid Retrieval:

Hybrid retrieval is a type of ensemble retrieval method that untilizes a dense search for vectors and a sparse search for keywords. It combines the strengths of both dense vector search and sparse keyword search to improve the relevance of the retrieved results.

Dense Vector Search: The vector store is then queried to find the most similar chunks based on cosine similarity between the query vector and the chunk vectors. This method captures semantic similarity and can retrieve relevant chunks even if they do not contain the exact keywords from the query.

Sparse Keyword Search: A traditional keyword based search is performed on the job description text to find chunks that contain the exact keywords from the user query. This method can capture specific terms and phrases that may be important for relevance, especially in cases where the query contains specific job titles, skills, or other keywords.

The results from both retrieval methods are then combined using Reciprocal Rank Fusion (RRF), which is a technique for combining ranked lists of results from different retrieval methods. The RRF score for a given chunk is calculated as follows:

$$RRFScore = \sum_{i=1}^{N} \frac{1}{k_i + 1} \tag{ii}$$

Where:

- N is the number of retrieval methods (in this case, 2: dense and sparse)
- k_i is the rank of the chunk in the results from the i-th retrieval method (starting from 0 for the top-ranked result)

The RRF score gives more weight to chunks that are ranked higher in either retrieval method, allowing for a more balanced combination of results. Top K chunks based on the RRF scores are then selected as candidate chunks for the next phase of reranking and generation.

But, for the reranker, **K + 1** chunks are selected to ensure enough candidates are available for reranking.

In this way, the biased generation from the dataset can be mitigated by ensuring relevant chunks are fetched.

2.2.1 Reranking Phase:

Reranking refers to the process of taking the candidate chunks retrieved from the previous phase and reordering them based on their relevance to the user query. This is typically done using a more computationally expensive model, such as a cross-encoder, that can capture finer, more granular semantic relationships between the query and the chunks.

Cross-Encoder:

A cross-encoder is a type of neural network architecture that takes two inputs (in this case, the user query and a candidate chunk) and processes them together to produce a relevance score. Unlike bi-encoders, which encode the query and the chunks separately, cross-encoders allow for interactions between the query and the chunk during the encoding process, which can lead to more accurate relevance scoring.

The cross-encoder architecture typically consists of a transformer based model that takes the concatenated query and chunk as input and produces a single output score that indicates the relevance of the chunk to the query. The figure 9 illustrates the architecture of a cross-encoder.

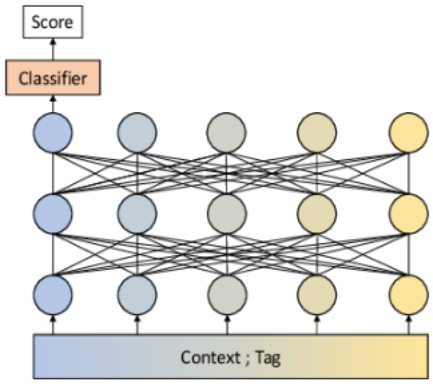

Figure 9: Architecture of a Cross-Encoder
Source: https://www.semanticscholar.org/paper/Cross-Encoding-as-Augmentation

The top K ranked passages after rerankin are then selected as the final context to be injected into the language model for generation. These passages are concatenated together with appropriate separators to form the context section of the prompt, ensuring that the most relevant information is provided to the model for generating a response to the user query.

2.3 Generation Phase:

This phase of the pipeline is responsible for taking the retrieved and reranked chunks of job descriptions as input and using them as context to generate a response to the user query.

It utilizes a language model to produce a concise, cited answer based on the provided context and the user query.

For this implementation, I used the Qwen2.5-3B-Instruct model from Hugging Face Transformers, which is a powerful language model designed for instruction following tasks. It is an open source, 2.5 billion parameter model that is optimized for generating high-quality responses based on provided instructions and context.

Hugging Face Transformers is a popular library for natural language processing that provides a wide range of pre-trained models and tools for working with them.

Prompt Design and Engineering:

Prompt design is a critical aspect of the generation phase, as it determines how the retrieved context and user query are presented to the language model to elicit the desired response. For this system, a system prompt was designed to instruct the model to answer only based on the retrieved context and to cite the relevant chunks in the response. The system prompt is structured as follows:

```
- answer as a precise job search assistant
- use only provided job listings
- do not invent companies or requirements
- say when context is insufficient
- be concise and structured
- include title, company, location, and level
- cite source numbers
```

This prompt is designed for hallucination mitigation and consistent output structure. These prompting decisions were refereced from big LLMs like ChatGPT and Claude, which have similar system prompts for grounded generation.

The retrieved and reranked chunks are then formatted and injected into the prompt as context, with clear separators between chunks to help the model distinguish between different pieces of information. The user query is also included in the prompt to provide the model with the specific question it needs to answer based on the provided context. The final prompt is then passed to the Qwen2.5-3B-Instruct model for generation.

2.4 Serving Phase:

Now that the retrieval and generation components are implemented, the next step is to serve the RAG system so that it can handle user queries in real-time. For this purpose, a FastAPI endpoint was built that provides an API endpoint for receiving user queries and returning generated responses.

FastAPI is a modern, python native web framework that is designed for building APIs quickly and efficiently. It is a restful framework that provides features such as automatic request validation, serialization, and documentation generation, making it an ideal choice for serving the RAG system.

Pydantic is a data validation library that is used in conjunction with FastAPI to define the expected structure of the request and response data. It is the automatic request validation provided by FastAPI and Pydantic ensures that the incoming user queries are properly structured and contain the necessary information for processing, which helps to prevent errors and improve the robustness of the API.

Langchain is a framework for building language model applications that provides tools and abstractions for working with language models, including prompt management, chaining of operations, and integration with various LLMs.

Using these libraries, the RAG system is served through a FastAPI application that listens for incoming POST requests at the '/api/query' endpoint. When a user sends a query to this endpoint, the API processes the request, performs retrieval and generation using the RAG pipeline, and returns the generated response in a structured JSON format.

The entire service Query-to-Response flow is as follows:

1. User sends JSON to `POST /api/query`.
2. FastAPI parses the body.
3. Pydantic validates query, `top_k`, and filters.
4. API checks `vector_store.is_ready`.
5. API logs query metadata.
6. `RAGPipeline.run` starts a timer.
7. Pipeline computes candidate fetch count.
8. `HybridRetriever.retrieve` embeds the query.
9. ChromaDB returns dense nearest neighbors.
10. Keyword retrieval may return sparse matches.
11. RRF may fuse dense and keyword results.
12. Reranker may rescore candidates.
13. `format_context` creates numbered evidence.
14. `build_answer_prompt` creates user prompt.
15. `LLMService.generate` invokes Hugging Face pipeline.
16. Pipeline catches LLM failures and falls back to a retrieval-only message if needed.
17. Raw candidates are converted to `JobChunk`.
18. Sources are deduplicated by job ID.
19. Retrieval metadata is assembled.
20. `QueryResponse` is returned to FastAPI.
21. FastAPI serializes the response as JSON.

The FastAPI also provides the Swagger UI for interactive testing and usage of the API, without the need for a dedicated frontend client.

CORS (Cross Origin Resource Sharing) is a protocol that allows web applications running on one domain to make requests to resources on a different domain. The CORS for this locally served API is configured permissively to allow requests from any origin.

`POST /api/query`

Input: `QueryRequest`.

Output: `QueryResponse`.

Internal logic:

1. Check vector store readiness.
2. Log query details.
3. Run RAG pipeline.
4. Convert pipeline failures into `HTTP 500`.
5. Return structured response.

`GET /api/health`

Input: none.

Output: `HealthResponse`.

Internal logic:

1. Report service status.
2. Report application version.
3. Report vector store readiness.
4. Report embedding model.
5. Report effective LLM provider and model.

Some quality of life features were also implemented as such as logging, error handling, and response formatting to improve the usability and maintainability of the API.